Memory Management in Linux

- Virtual Memory, Demand Paging, and Page Fault Analysis
- Operating Systems (B24CS2T04)
- Self-Learning Practice – Topic 5

Agenda

1. Why Memory Management Matters
2. Virtual Memory in Linux
3. How Demand Paging Works
4. Live Demo: Memory-Intensive Program
5. Page Fault Comparison using `vmstat`
6. Key Takeaways

Why Memory Management?

The Problem:

- Physical RAM is limited (4 GB, 8 GB, 16 GB…)
- Multiple processes run simultaneously
- Each process thinks it owns all memory

The Solution: Virtual Memory

- Every process gets its own private address space
- The OS + hardware map virtual → physical transparently
- Only actively-used pages consume RAM (demand paging)

Imagine a library where books are only fetched from the warehouse when someone asks for them — that’s demand paging.

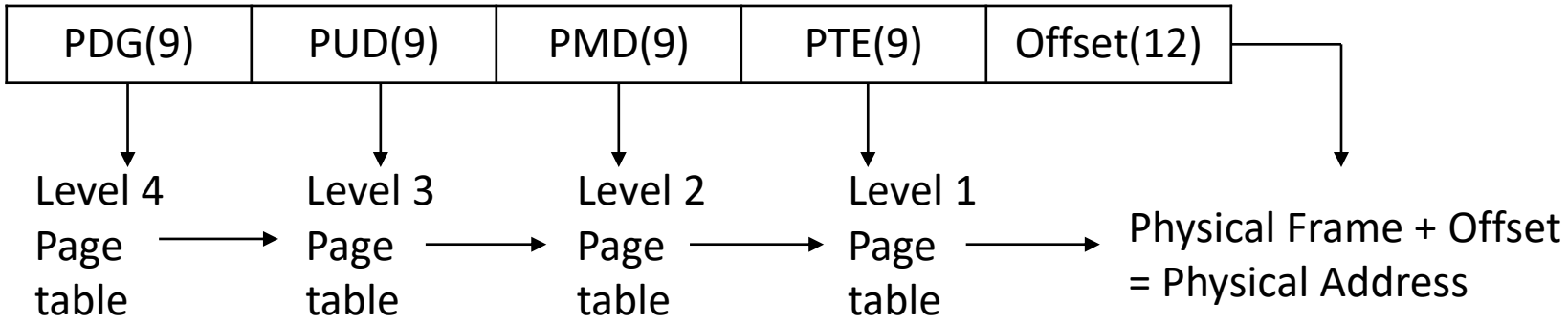
Virtual Address Space Layout

Kernel Space	(not accessible from user mode)
Stack ↓	(grows downwards)
Memory mapped	(shared libs, mmap)
Heap ↑	(grows upwards – malloc)
BSS	(uninitialized globals)
Data	(initialized globals)
Text	program code

- Each section is a **Virtual Memory Area (VMA)**
- Kernel tracks them with vm_area_struct linked list
- - All VMAs for a process live inside mm_struct — the per-process memory descriptor (also holds the pgd pointer to the top-level page table)
- /proc/<pid>/maps shows the actual layout — **DEMO**: run cat /proc/self/maps

Page Tables & Translation

Virtual Adress (48 bits used on x86 – 64)



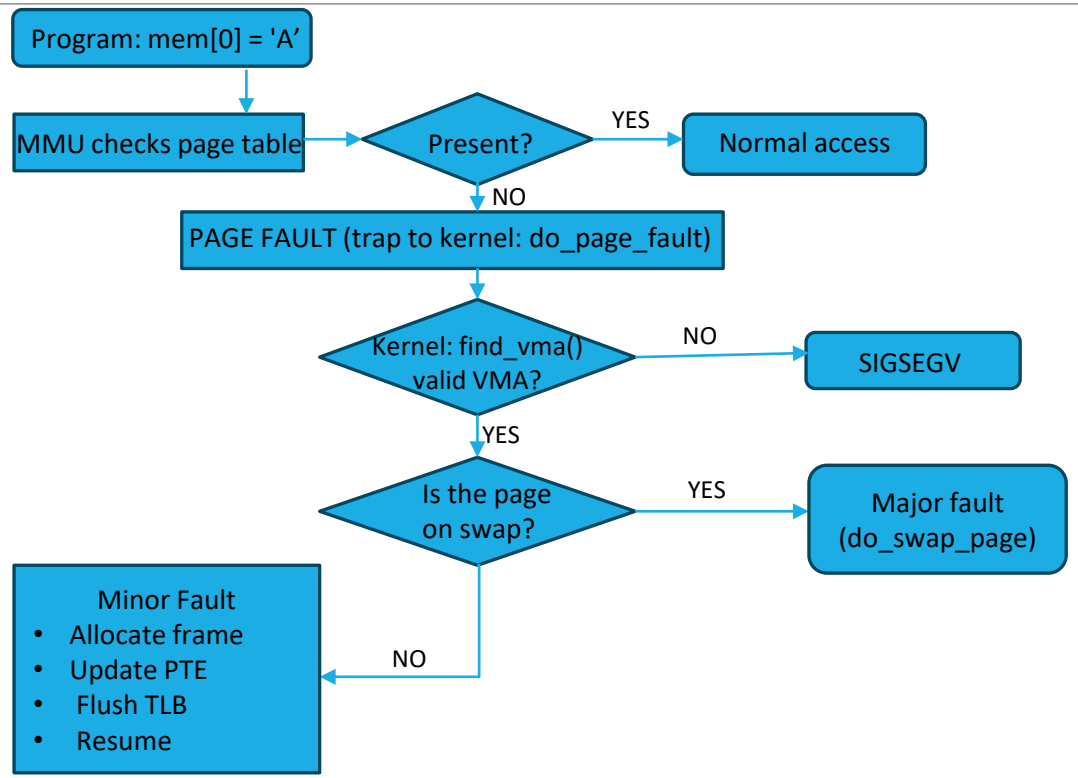
- TLB (Translation Lookaside Buffer) caches recent translations
- TLB miss → full 4-level walk (expensive!)
- This is why access pattern matters

What is a Page Fault?

Type	Cause	Cost
Minor fault	Page exists but no physical frame yet (first access)	~microseconds
Major fault	Page data must be read from disk/swap	~milliseconds (1000× slower)
Invalid fault	Access to unmapped address	→ SIGSEGV (crash)

Key insight: `malloc()` does NOT allocate physical memory. It only reserves virtual addresses. Physical frames are allocated on first access via minor faults.

Demand Paging Flow



Note

On a first read (not write), the kernel maps the global empty_zero_page instead of allocating a new frame — a read-optimization. A real physical frame is allocated only on the first write (copy-on-write from the zero page).

This is the beauty of demand paging — the kernel is lazy in the best way. It never does work upfront that might not be needed.

Our Demo Program Design

demand_paging_demo.c — Three phases:

Phase	What happens	Expected faults
1. malloc(512 MB)	Reserve virtual space	~0 faults
2. Touch all pages	Write one byte per page	~131,072 minor faults
3. Re-touch pages	Write again	~0 faults (already in RAM)

Three access patterns to compare:

- **Sequential** — page 0, 1, 2, 3 … (best locality)
- **Random** — Fisher-Yates shuffled order (worst TLB behaviour)
- **Stride-64** — every 64th page, then wrap (moderate)

We measure faults by reading /proc/self/stat (fields 10 & 12)

Our Demo Program Design

demand_paging_demo.c — Three phases:

Phase	What happens	Expected faults
1. malloc(512 MB)	Reserve virtual space	~0 faults
2. Touch all pages	Write one byte per page	~131,072 minor faults
3. Re-touch pages	Write again	~0 faults (already in RAM)

```c
/* --- Phase 1 : Allocate (no pages are physically mapped
yet) --- */
long min0, maj0;
read_page_faults(&min0, &maj0);
printf("[Phase 1] malloc(%zu MB) — only virtual addresses
reserved.\n", size_mb);

volatile char *mem = (volatile char *)malloc(size);
if (!mem) { perror("malloc"); return 1; }

long min1, maj1;
read_page_faults(&min1, &maj1);
printf("  Page faults after malloc  : minor=%ld  major=%ld\n",
       min1 - min0, maj1 - maj0);
printf("  (Expect ~0 because no physical frames allocated
yet.)\n\n");
```

Our Demo Program Design

demand_paging_demo.c — Three phases:

Phase	What happens	Expected faults
1. malloc(512 MB)	Reserve virtual space	~0 faults
2. Touch all pages	Write one byte per page	~131,072 minor faults
3. Re-touch pages	Write again	~0 faults (already in RAM)

```c
/* --- Phase 2 : Touch pages (demand paging occurs here) --------
*/
long min2, maj2;
read_page_faults(&min2, &maj2);
printf("[Phase 2] Touching pages — demand paging in action.\n");

struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);

if      (strcmp(pattern, "seq")    == 0) access_sequential(mem,
size);
else if (strcmp(pattern, "random") == 0) access_random(mem, size);
else if (strcmp(pattern, "stride") == 0) access_stride(mem, size);
else {
    fprintf(stderr, "Unknown pattern '%s'. Use: seq | random |
    stride\n", pattern);
    free((void *)mem);
    return 1;
}

clock_gettime(CLOCK_MONOTONIC, &t1);

long min3, maj3;
read_page_faults(&min3, &maj3);

double elapsed = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.
tv_nsec) / 1e9;

printf("\n  Page faults during access  : minor=%ld  major=%ld\n",
        min3 - min2, maj3 - maj2);
printf("  Time elapsed                : %.4f seconds\n", elapsed);
printf("  Approx faults / second      : %.0f\n\n",
        (min3 - min2) / elapsed);
```

Our Demo Program Design

demand_paging_demo.c — Three phases:

Phase	What happens	Expected faults
1. malloc(512 MB)	Reserve virtual space	~0 faults
2. Touch all pages	Write one byte per page	~131,072 minor faults
3. Re-touch pages	Write again	~0 faults (already in RAM)

```c
/* --- Phase 3 : Re-access (pages already in memory) ------------
*/
printf("[Phase 3] Re-touching all pages (should cause ~0 new
faults).\n");
long min4, maj4;
read_page_faults(&min4, &maj4);

access_sequential(mem, size);                /* always
sequential */

long min5, maj5;
read_page_faults(&min5, &maj5);
printf("  Page faults on re-access   : minor=%ld  major=%ld\n",
        min5 - min4, maj5 - maj4);
printf("  (Expect ~0 because frames are already mapped.)\n\n");
```

Our Demo Program Design

Three access patterns to compare:

- **Sequential** — page 0, 1, 2, 3 … (best locality)
- **Random** — Fisher-Yates shuffled order (worst TLB behaviour)
- **Stride-64** — every 64th page, then wrap (moderate)

We measure faults by reading /proc/self/stat (fields 10 & 12)

```c
/*
 * Sequential access – touches every page exactly once in order.
 * Expected: a burst of minor page faults equal to total_pages.
 */
static void access_sequential(volatile char *mem, size_t size)
{
    printf("[SEQ] Touching every page sequentially ...\n");
    for (size_t off = 0; off < size; off += page_size)
        mem[off] = (char)(off & 0xFF);
}
```

Our Demo Program Design

Three access patterns to compare:

- **Sequential** — page 0, 1, 2, 3 … (best locality)
- **Random** — Fisher-Yates shuffled order (worst TLB behaviour)
- **Stride-64** — every 64th page, then wrap (moderate)

We measure faults by reading /proc/self/stat (fields 10 & 12)

```c
/*
 * Random access – touches every page exactly once in random order
 * using a Fisher-Yates (Knuth) shuffle of page indices.
 * Expected: same total faults as sequential, but more TLB misses.
 */
static void access_random(volatile char *mem, size_t size)
{
    size_t total_pages = size / page_size;
    printf("[RND] Touching %zu pages in random order (Fisher-Yates shuffle) ...\n", total_pages);

    /* Build an index array and shuffle it so every page is visited once */
    size_t *indices = (size_t *)malloc(total_pages * sizeof(size_t));
    if (!indices) { perror("malloc indices"); return; }
    for (size_t i = 0; i < total_pages; i++) indices[i] = i;

    srand((unsigned)time(NULL));
    for (size_t i = total_pages - 1; i > 0; i--) {
        size_t j = (size_t)rand() % (i + 1);
        size_t tmp = indices[i];
        indices[i] = indices[j];
        indices[j] = tmp;
    }

    for (size_t i = 0; i < total_pages; i++)
        mem[indices[i] * page_size] = (char)(indices[i] & 0xFF);

    free(indices);
}
```

Our Demo Program Design

Three access patterns to compare:

- **Sequential** — page 0, 1, 2, 3 … (best locality)
- **Random** — Fisher-Yates shuffled order (worst TLB behaviour)
- **Stride-64** — every 64th page, then wrap (moderate)

We measure faults by reading /proc/self/stat (fields 10 & 12)

```c
/*
 * Stride access – touches every N-th page, then wraps around.
 * Useful for showing how locality affects TLB / page-fault cost.
 */
static void access_stride(volatile char *mem, size_t size)
{
    size_t total_pages = size / page_size;
    size_t stride = 64;                          /* skip 64 pages   */
    printf("[STRIDE] Touching pages with stride=%zu ...\n", stride);
    for (size_t pass = 0; pass < stride; pass++)
        for (size_t pg = pass; pg < total_pages; pg += stride)
            mem[pg * page_size] = (char)(pg & 0xFF);
}
```

LIVE DEMO

Terminal 1

```
miyan@Galaxy:~$ cd code/
miyan@Galaxy:~$ gcc -O0 -o demand_paging_demo demand_paging_demo.c

miyan@Galaxy:~$ ./demand_paging_demo 512 seq
```

Terminal 2

```
miyan@Galaxy:~$ vmstat 1
```

```
miyan@Galaxy:~$ ./demand_paging_demo 512 seq

============================================
  Demand Paging Demonstration
============================================
  Page size       : 4096 bytes
  Allocation      : 512 MB  (131072 pages)
  Access pattern  : seq
============================================

[Phase 1] malloc(512 MB) – only virtual addresses reserved.
  Page faults after malloc  : minor=0  major=0
  (Expect ~0 because no physical frames allocated yet.)

[Phase 2] Touching pages – demand paging in action.
[SEQ] Touching every page sequentially ...

  Page faults during access  : minor=131072  major=0
  Time elapsed               : 0.3518 seconds
  Approx faults / second     : 372568

[Phase 3] Re-touching all pages (should cause ~0 new faults).
[SEQ] Touching every page sequentially ...
  Page faults on re-access   : minor=0  major=0
  (Expect ~0 because frames are already mapped.)

============================================
  Summary
============================================
  Total minor faults  : 131076
  Total major faults  : 0
============================================
```

vmstat Output During Sequential Demo

Baseline (before demo starts):

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0      0 6234512  45672 1024768   0    0     0     0   85  150  0  0 100 0  0
 0  0      0 6234480  45672 1024768   0    0     0     0   82  145  0  0 100 0  0
```

During Phase 2 (sequential, 512 MB):

```
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0 5972224  45672 1024768   0    0     0     0  285  420 12 24 64  0  0
 1  0      0 5710208  45672 1024768   0    0     0     0  320  480 15 25 60  0  0
 1  0      0 5448192  45672 1024768   0    0     0     0  310  460 14 26 60  0  0
 1  0      0 5186304  45672 1024768   0    0     0     0  305  455 13 27 60  0  0
```

After demo finishes (memory freed):

```
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0      0 6234128  45672 1024768   0    0     0     0   90  160  1  1 98  0  0
 0  0      0 6234256  45672 1024768   0    0     0     0   84  148  0  0 100 0  0
```

vmstat Explained

```
procs ----memory----- --swap-- ---io--- -system- -----cpu-----
r  b    swpd  free     si so     bi bo   in  cs   us sy id wa
```

Column	What to watch	Normal	During demo
`free`	Free RAM (KB)	~6 GB	Drops by ~512 MB
`si`	Swap in (KB/s)	0	0 (if RAM sufficient)
`so`	Swap out (KB/s)	0	0 (if RAM sufficient)
`sy`	System CPU %	<5%	20-30% (fault handling)

Automated monitoring: vmstat_monitor.sh captures vmstat snapshots before, during, and after the demo run and writes a summary log — useful for including in reports.

To trigger swap activity: allocate more than available RAM

./demand_paging_demo 7000 seq *# on 8 GB machine*

 # Now vmstat will show si/so > 0

Comparison Results

Pattern	Minor Faults	Major Faults	Time (s)
Sequential	~131,072	0	~0.35
Random	~131,072	0	~0.85
Stride-64	~131,072	0	~0.55

(This table is the output of compare_page_faults.sh — saved to page_fault_comparison.txt)

Why same faults but different speed?

- **Same faults:** All patterns touch every page once → same number of first-access faults
- **Different speed:** TLB and cache locality!
  - Sequential: TLB entries reused within contiguous region; CPU prefetch works
  - Random: TLB thrashes; every access likely misses L1/L2/L3 cache
  - Stride-64: TLB partially effective; some prefetch benefit

Bonus: compare_page_faults.sh also captured per-pattern vmstat logs (vmstat_run_seq.log, vmstat_run_random.log, vmstat_run_stride.log) — you can compare sy and free deltas across patterns to see the system-level impact

Page Replacement

When RAM is full, the kernel must **evict** pages to make room.

Linux uses a two-list LRU approximation:

- Pages start on the inactive list
- If accessed again → promoted to active list
- kswapd daemon periodically moves active → inactive
- Active list is kept at ~2/3 of total page cache (modified 2Q algorithm)
- Eviction candidates come from the inactive list tail

Thrashing

Thrashing = working set > physical memory → constant page-in/page-out

Symptoms:

- CPU idle time is high (waiting for I/O)
- `vmstat` shows `si/so` constantly > 0
- System becomes unresponsive

Solutions:

- Add more RAM
- Reduce the number of concurrent processes
- Use cgroups to limit per-process memory
- Working set size estimation (PFF — Page Fault Frequency)

SDG Connection

SDG	Connection
SDG 9 (Industry & Infrastructure)	Virtual memory enables cloud infrastructure
SDG 12 (Responsible Consumption)	Demand paging = allocate only what you use
SDG 13 (Climate Action)	Efficient memory → fewer servers → less energy
SDG 4 (Quality Education)	Peer-taught video class; hands-on experimentation

Key Takeaways

1. **Virtual memory** gives every process its own isolated address space
2. **malloc does NOT allocate physical RAM** — it's demand-paged
3. **Minor faults** (first access) are fast; **major faults** (swap) are slow
4. **Access pattern** doesn't change fault count but greatly affects TLB/cache performance
5. **vmstat** is the go-to tool for monitoring memory and swap activity
6. Understanding these concepts is essential for writing performant systems software

Discussion & Further Exploration

Questions?

Further exploration:

- `perf stat -e page-faults,dTLB-load-misses ./demand_paging_demo 512 seq`
- Read Chapters 9–10 of Silberschatz (10th ed.)
- OSTEP Chapters 18–22: https://pages.cs.wisc.edu/~remzi/OSTEP/